

Pass 3 - Inline-Editable Continuity Signals

CC-TX-2026-05-28-003 | 2026-05-28 | Active Canon

Layer: Session Workspace / Continuity Rail

From: Paula Accord

To: Cody Valle

Authorized By: Brandon Hatfield, LPC

Summary

Pass 3 made the continuity signals in the right rail inline-editable within the active session workspace. A single new VaultStore method handles the mutation.

'ContinuitySignalRow' gained optional edit mode via an 'onUpdate' closure; read-only display (home dashboard, empty states) is unchanged. Three new XCTest methods cover persistence, field isolation, and timestamp correctness. Cody verified the pass with 'swift build' and 'Tools/run-tests.sh'; 46 tests passed with 0 failures.

Body

ID: CC-TX-2026-05-28-003

Status: Active Canon

Authorized By: Brandon Hatfield, LPC

Source Issue: Paula-originated log

The continuity signal data path in 'ClinicalSession' was already defined:

'continuitySignals: [ContinuitySignal]', where each 'ContinuitySignal' carries a stable 'id: UUID', 'kind: SignalKind', 'title: String', and 'detail: String'. What was missing was a mutation route through the store and an edit surface in the rail.

VaultStore.swift received one new method, 'updateContinuitySignal(sessionID:signal:)', placed immediately before 'completeSession'. It locates the session by ID, locates the signal within that session by signal ID, replaces the signal value in-place, stamps 'session.updatedAt', and calls 'save()'. No audit event is recorded - signal edits follow the same silent-save convention as draft field edits. The guard pattern matches 'updateSessionDraft': two 'guard let index' lookups, early return on miss.

Views.swift required changes at four points.

'ContinuitySignalRow' was refactored from four decomposed string parameters ('eyebrow', 'title', 'detail', 'systemImage') to a single 'signal: ContinuitySignal' parameter plus an optional 'onUpdate: ((ContinuitySignal) -> Void)?' closure. Local '@State' properties

'editTitle' and 'editDetail' are seeded from the signal in a custom 'init'. When 'onUpdate' is provided, both fields render as inline 'TextField' controls with '.plain' style - 'detail' uses 'axis: .vertical' with 'lineLimit(1...4)' for natural growth. When 'onUpdate' is nil, the existing 'Text' display is preserved. In both 'onChange' handlers, both local state fields are used when reconstructing the updated signal: 'updated.title = editTitle' in the detail handler, 'updated.detail = editDetail' in the title handler. This prevents stale prop values from overwriting concurrent edits.

'ContinuitySignalStack' gained an optional 'var onUpdate: ((ContinuitySignal) -> Void)?' that it passes through to each 'ContinuitySignalRow'.

'RightContinuityRail' gained an optional 'var onUpdateSignal: ((ContinuitySignal) -> Void)?' that it passes into 'ContinuitySignalStack.onUpdate'.

'ActiveSessionWorkspace' now passes the closure into its 'RightContinuityRail' call:

```
""swift
RightContinuityRail(
palette: palette,
session: liveSession,
onUpdateSignal: { signal in
vaultStore.updateContinuitySignal(sessionID: session.id, signal: signal)
}
)
""
```

The home dashboard's 'RightContinuityRail' call is unchanged - no 'onUpdateSignal' argument, so signals remain read-only there as intended.

****VaultStoreTests.swift**** received three new test methods in a '// MARK: - Continuity Signal' section following the existing Complete Session block:

'testUpdateContinuitySignalPersistsAcrossLockUnlock' edits the first fixture signal's title, lock/unlocks, and verifies the new title survives the round-trip.

'testUpdateContinuitySignalPreservesOtherFields' confirms that a title-only edit leaves 'detail' and 'kind' unchanged, guarding against accidental field erasure during the struct reconstruction in 'onChange'.

'testUpdateContinuitySignalUpdatesSessionTimestamp' confirms that 'session.updatedAt' is

equal to or later than the value observed before the edit.

All three tests guard on 'session.continuitySignals.first' - the first fixture session carries three signals, so this assumption holds against the current fixture data.

Decisions

- Signal edits use the same silent-save pattern as draft edits; no audit event.
- 'ContinuitySignalRow' uses a custom 'init' to seed '@State' from the signal prop rather than using '@Binding'. This matches the 'ClinicalDraftPanel' pattern and avoids requiring callers to manage state externally.
- The read/write distinction is controlled entirely by whether 'onUpdate' is nil or non-nil - no separate 'isEditable' flag.
- 'TextField(axis: .vertical)' is valid on macOS 14 (the project target). Detail fields grow naturally up to four lines.
- Home dashboard continuity signals remain read-only; only the active session workspace enables editing. The closure threading handles this without forking the component.
- 'ContinuitySignal.kind' is not editable. Kind is set at fixture creation and treated as session metadata. If mutability is needed in a future pass, it should be a deliberate scoped addition.

Next Actions

- Paula Pass 4: copy-to-clipboard and '.txt' export from 'GeneratedDraftPreview', following the 'exportBackup()' pattern.
- Housekeeping pending from prior session: 'paula_accord_portrait.png' and 'paula_accord_philosophy.md' remain in project root - Cody to delete in Finder when convenient.